LakehouseGuard

A contract-driven, self-healing lakehouse with an AI incident agent. Data you can trust, incidents you can explain.



contracts · quarantine · observability · RCA

Rushikesh Wavhal

Azure Data Engineer · Microsoft Fabric · Databricks · PySpark

Pipelines pass tests. Data still breaks

The failures that hurt are rows that look valid but are quietly wrong: late, duplicated, renamed, rescaled.

Build-time tests do not see production data. Dashboards go wrong before anyone knows why.



Silent bad data

Valid-looking rows that are wrong reach Gold and the dashboards unnoticed.



Alert floods

One schema change fires five alerts. Nobody knows which one matters.



No root cause

Hours spent asking which source, which column, which batch, which fix.

Four layers of defence



Data contracts

YAML contracts gate Bronze to Silver. Rows that fail go to Quarantine with the exact rules they broke.

Never a silent drop



Self-healing

Fixes only the unambiguous: whitespace, casing, aliases, currency-formatted numbers.

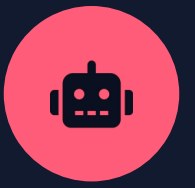
Over 10% healed raises an incident



Observability

Freshness, volume, nulls, schema and silent distribution drift (PSI).

Baseline learns only from healthy runs



AI incident agent

Collapses alerts to one root cause, ranks hypotheses, retrieves the runbook.

No API key needed

Every batch takes the same path



Replay-safe

Same batch_id overwrites its partitions and Gold rebuilds, so retries and backfills never double count.

Clear rules for what happens next

Situation	What the platform does
A row breaks a rule	Row goes to Quarantine with every rule it broke; the rest of the batch continues
Required column missing	Whole batch blocked, nothing reaches Silver, critical incident
More than 50% of rows fail	Circuit breaker holds the batch: a heavily filtered partial batch would bias Gold KPIs
Value is repairable without guessing	Auto-repaired and counted; over 10% repaired raises high_heal_rate
Data is valid but behaves differently	Published and alerted: contracts block, observability alerts
Batch raised any incident	Never updates the baseline, so bad data cannot teach the platform that bad is normal

Nine ways to break it, all injected on purpose

1
null_burst
15% of customer_id nulled

Contract: not_null

2
schema_drift
amount renamed to order_amount

Schema break, batch blocked

3
type_drift
N/A and -- in a numeric field

Contract: type

4
duplicate_flood
25% of orders delivered twice

Contract: unique

5
volume_drop
Only 8% of rows arrive

Observability: robust z

6
outlier_amount
5% of amounts multiplied by 400

Contract: range

7
stale_data
Timestamps shifted 6 hours

Freshness SLA

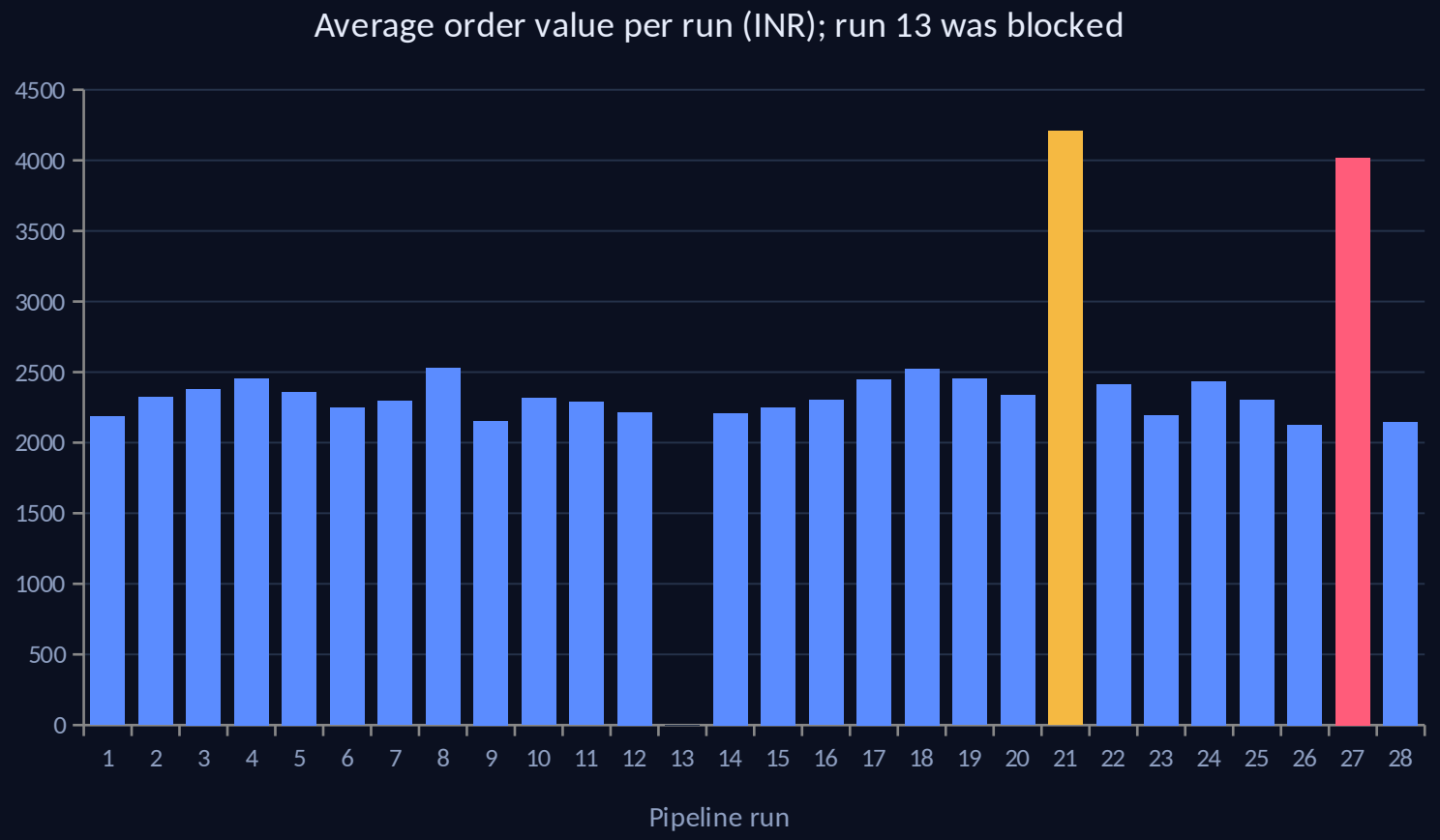
8
format_drift
₹1,234.50 and lowercase cities

Auto-healed, then flagged

9
silent_drift
All amounts x1.8, every row valid

Observability: PSI

Every row valid. The data is still wrong



100%

of rows passed the contract in run 27

1.73x

mean order value: INR 2,324 to 4,019

0.48

PSI vs reference, threshold 0.2: only observability sees it

Run 21 (amber): a few 400x outliers stayed under the contract's global max and still lifted the average. See limits.

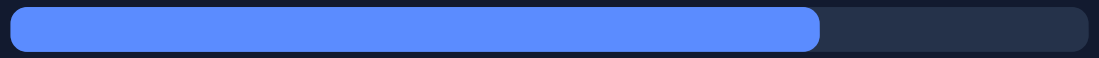
From alerts to one root cause

SCHEMA_BREAK · CRITICAL

Required column(s) missing: amount

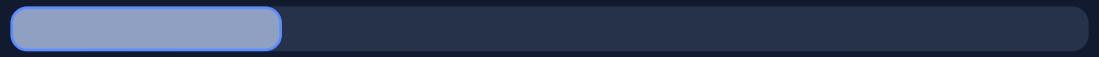
Most likely causes

Upstream renamed amount to order_amount without a contract version bump



75%

Source API or extraction query version changed



25%

Evidence: a column vanished while a new one appeared in the same batch.

Symptoms folded in: schema_drift. One alert to act on instead of two.

Blast radius

1,225 of 1,225 rows

silver, gold fact and KPI mart held back

Runbook (retrieved by TF-IDF)

Ask the producer to roll back or bump the contract; replay the batch with the same batch_id.

Suggested starting point

```
df.rename(columns={'order_amount':
'amount'})
```

Measured, not claimed

100%

180 injected failure batches detected with the expected incident type

0 of 100

false positives on healthy batches

29%

fewer alerts after causal triage (254 to 180)

~33k

rows per second, single-process pandas engine

Read this before quoting the numbers

Synthetic, seeded data (9 modes x 20 seeds). The failure modes and the detectors share an author, so 100% means it catches what it was designed against, not everything. The agent's top-1 root cause is checked against the injected cause. Row-level recall on 400x outliers is 73% because some stay under the global range.

Built like a product, not a notebook

98

tests

100%

line coverage

2

Python versions in CI



CI/CD

GitHub Actions: lint, tests on 3.11 and 3.12, coverage gate, benchmark smoke test



One command

docker compose up serves the dashboard; pip install and lakehouseguard demo also work



Live dashboard

GitHub Pages rebuilds a self-contained HTML dashboard from the pipeline on every push



Documented decisions

Six ADRs, eight runbooks, architecture, Azure mapping and an interview guide

Coverage measured with a stdlib tracer here; CI enforces 90% with coverage.py.

Same ideas, production services

LakehouseGuard	Azure Databricks	Microsoft Fabric	Azure services
Bronze / Silver / Gold	Delta + Unity Catalog	Lakehouse in OneLake	ADLS Gen2
Contract gate	Lakeflow expectations	Notebook, Dataflow Gen2	ADF data flow assert
Quarantine	Delta quarantine table	Lakehouse table	ADLS container
Observability	Lakehouse Monitoring	Real-Time Intelligence	Azure Monitor
Alerts	Workflows alerts	Fabric Activator	Logic Apps, Teams
Lineage	Unity Catalog lineage	Purview	Microsoft Purview
Orchestration	Workflows	Fabric pipelines	ADF, Airflow
Dashboard	Databricks SQL	Power BI	Power BI

examples/spark_engine.py expresses the contract gate as PySpark + Delta (idempotent per-batch write).

What it is not, and what comes next

Honest limits

- Pandas core is single-process; Spark is a reference example
- A global max lets some outliers through (73% row-level recall)
- The agent is a playbook plus retrieval, not a learner
- Confidence is a normalised weight, not a probability

Roadmap

- Per-segment range rules and robust outlier detection
- Embedding retrieval for runbooks
- Incremental Gold and dbt-core marts
- OpenLineage to Marquez, auto-created tickets

Let's make data trustworthy

Clone it, run lakehouseguard demo, and break it yourself in 60 seconds.

Rushikesh Wavhal

github.com/AnyalstRushi/lakehouseguard

rushiwavhal161718@gmail.com

Pune · Open to relocate · Available in 15 days